

5주차

📅 날짜	2026년 10월 6일
☰ 다중 선택	
≡ 텍스트	파머완 4.9 & 캐글 notebook 1,2

4장 분류

9. 분류 실습 - 캐글 산탄데르 고객 만족 예측

- 370개 피처로 주어진 데이터 세트 기반 → 고객 만족 여부 예측
- 산탄데르 은행이 캐글에 경연 의뢰한 데이터. 피처 이름 모두 익명 처리 → 이름만으로 어떤 속성 추정 불가
- 클래스 레이블 명 : TARGET
값 1 : 불만 가진 고객 / 값 0 : 만족한 고객
- 모델 성능 평가 : 정확도 수치 < **ROC-AUC** 더 적합
→ 대부분이 만족이고, 불만족인 데이터는 일부일 것이기 때문

1) 데이터 전처리

1. 필요한 모듈 로딩 & 학습데이터 DataFrame으로 로딩
2. 피처 타입, Null 값 확인 → float형 피처 111개, int형 피처 260개, Null값 없음.
레이블인 TARGET 속성 값 분포 → 대부분 만족, 불만족 고객 - 4%
3. 각 피처 값 분포 확인 : describe() 매소드
4. 클래스 데이터 세트와 피처 데이터 세트 분리 → 별도의 데이터 세트로 저장
이때 비대칭한 데이터 세트이므로 ⇒ 클래스인 TARGET 값 분포도가 학습&테스트 데이터 세트에 모두 비슷하게 추출 됐는지 확인
5. 원본 데이터 세트 ⇒ 학습 데이터 세트, 테스트 데이터 세트 분리

2) XGBoost 모델 학습과 하이퍼 파라미터 튜닝

XGBoost 학습 모델 생성 + ROC-AUC 평가

- 사이킷런 래퍼인 XGBClassifier 기반 학습 수행

- XGBClassifier의 `eval_metric = 'auc'`

XGBoost 하이퍼 파라미터 튜닝

HyperOpt 이용 - 베이지안 최적화 기반으로 수행

1. 하이퍼 파라미터 검색 공간 설정
2. 목적 함수 생성 - **3 Fold 교차검증 이용**
: 평균 ROC-AUC값 반환하되 -1 곱해서 $\Rightarrow$ 최대 ROC-AUC 값 = 최소 반환값 되도록
교차 검증 시, **KFold 클래스 이용**
: XGBoost 조기 중단 & 검증 데이터 성능 평가 위해
직접 학습&검증 데이터 세트 추출 + 교차 검증 횟수만큼 학습과 성능 평가 수행
3. `fmin()` 함수 호출해서 `max_eval = 50`회 만큼 반복 $\rightarrow$ 최적의 하이퍼 파라미터 도출
(50회 만큼 교차 검증 반복하여 학습+평가)
4. 도출된 최적 하이퍼 파라미터 기반으로 XGBClassifier 재학습시키고 테스트 데이터 세트에서 ROC-AUC 측정 $\Rightarrow$ **ROC-AUC 개선됨**

단점

- XGBoost가 GBM보단 빠르지만 GBM 기반이라 수행시간 많이 요구됨
- 앙상블 계열 알고리즘은 하이퍼 파라미터 튜닝 통한 성능 수치 개선 급격하게 되는 경우 많지 **×**

3) LightGBM 모델 학습과 하이퍼 파라미터 튜닝

학습 수행 + ROC-AUC 측정

앞의 XGBoost 데이터 세트 기반으로 학습 수행 $\rightarrow$ XGBoost 보다 학습 걸리는 시간 단축됨

하이퍼 파라미터 튜닝 수행

튜닝 대상만 달라짐. 과정 거의 동일 $\Rightarrow$ ROC-AUC 개선 **×**

LightGBM은 학습시간 상대적으로 빠름 $\rightarrow$ 다른 추가적인 하이퍼 파라미터 적용해서 튜닝 수행 가능

캐글 노트북

Decision Tree And Random Forest Classifier Models

1. 개요

- 목표 : 의사결정나무(DT), 랜덤포레스트(RF) 적용 → 혼동행렬(Confusion Matrix)로 평가
- 데이터 : Drug Classification (`drug200.csv`)
 - Feature : Age, Sex, BP, Cholesterol, Na_to_K
 - Target : Drug (DrugA / B / C / X / Y → 5개 클래스)
- 흐름 : 데이터 확인 → 시각화 → 전처리 → DT → RF → 평가

2. 데이터 확인 & 시각화

- `data.info()` → 컬럼 타입 / 결측 확인
- `data.describe()` → 기초통계 (max, min, std)
- `data.corr()` + `sns.heatmap()` → 상관관계 시각화
- `sns.pairplot(data, hue="Drug")` → 변수 쌍별 분포를 약물별 색으로 확인
- `value_counts()` → 범주형 컬럼 값 분포 확인
- plotly 사용 : `px.bar`, `go.Pie`, `px.scatter` (Na_to_K vs Age, 약물별 색)

3. 전처리 (Data Preparing)

- **!** 문자열 데이터는 학습 불가 → 정수 인코딩 필요
- 원본 보호 위해 CSV 재로딩 후 변환

컬럼	변환 규칙
Sex	F=1, M=0
BP	LOW=2, NORMAL=1, HIGH=0
Cholesterol	HIGH=1, NORMAL=0
Drug	DrugY=4, X=3, A=2, C=1, B=0

- 변환 후에도 object 타입이 남은 컬럼 → `.astype({'BP': int, 'Drug': int})`
- `x_data = dataclass.drop(["Drug"], axis=1)` → 입력(X)
- `y_data = dataclass.Drug.values` → 정답(y)
- `train_test_split(x, y, test_size=0.3, random_state=1)` → train 70% / test 30%

4. Decision Tree Classifier

- 개념
 - 지도학습 (분류 + 회귀 가능, 주로 분류에 사용)
 - 트리 구조 : 내부 노드 = feature / 가지 = 결정 규칙 / 리프 노드 = 결과
- 핵심 코드 : `DecisionTreeClassifier()` → `.fit()` → `.predict()` → `accuracy_score()`
- 주요 파라미터
 - `criterion` : 분할 기준 → `'gini'` / `'entropy'`
 - `max_depth=3` : 트리 깊이 제한
 - `random_state` : 결과 재현용
- `.score(x, y)` = 정확도 → train / test 점수 비교로 과적합 확인

모델	Train	Test
기본 DT	-	0.9667
gini (depth=3)	0.9143	0.9000
entropy (depth=3)	0.9143	0.9000

→ gini와 entropy는 이 데이터에서 결과가 동일

5. Random Forest Classifier

- 개념
 - 지도학습 (분류 + 회귀 가능)
 - **앙상블 학습** 기반 : 여러 분류기를 결합 → 복잡한 문제 해결 + 성능 향상
- 핵심 코드 : `RandomForestClassifier(n_estimators=100, random_state=0)`
 - `n_estimators` = 트리 개수
- 파라미터 탐색 (for문 + 그래프)
 - ① `random_state` 0~9 반복 → train/test 점수를 `sns.lineplot` 으로 비교 → 최적값 **5**
 - ② `n_estimators` [10~100] 반복 → 최적값 **100**
- 결과

모델	Test
기본 RF	0.95
n_estimators=100	0.95
최적 (n=100, rs=5)	0.9667 (Train 1.0)

6. 평가 : Confusion Matrix

- 정의
 - 정답을 아는 test 데이터에서 분류 모델의 예측 결과를 **표로 나타낸 것**
 - 평가지표(performance metrics)로 모델 성능을 확인하는 단계
- 핵심 코드
 - `confusion_matrix(y_true, y_pred)` → 행렬 생성
 - `sns.heatmap(cm, annot=True, fmt=".0f")` → 시각화
 - 축 : x = `y_pred` (예측) / y = `y_true` (실제)
- 적용 : 기본 DT / gini DT / 최적 RF 3개 모델

Beginner Friendly CATBOOST with OPTUNA

1. 개요

- 목표 : 분류 문제의 **End-to-End ML 프로젝트** → CatBoost + Optuna 튜닝
- 데이터 : Heart Failure Prediction
 - Feature : Age, Sex, ChestPainType, RestingBP, Cholesterol, FastingBS, RestingECG, MaxHR, ExerciseAngina, Oldpeak, ST_Slope
 - Target : **HeartDisease** (1=심장병 / 0=정상)
- 흐름 : 문제·지표 정의 → EDA → 모델 비교 → CatBoost → Optuna 튜닝 → Feature Importance

2. 문제 정의 & 평가지표

- 이진 분류 문제
- Target 비율 : 심장병 약 55% (508명) / 정상 약 45% (410명) → 거의 균형
- 균형 데이터이므로 **Accuracy**를 지표로 사용

3. EDA

- 기본 점검

- `df.info()` → 타입 / `df.duplicated().sum()` → 중복 없음
- 직접 만든 `missing()` 함수 → 결측 없음
- `select_dtypes('number')` / `('object')` → 수치형 / 범주형 분리
- `.skew()` + 임계값 0.75 → 왜도 크지 않음 (정규분포에 가까움)

- 핵심 인사이트

- 수치형 ↔ Target 상관은 전반적으로 약함
 - Oldpeak : (+) 상관
 - MaxHR : (-) 상관
 - Cholesterol : (-) 상관 (의외의 결과)
- 남성이 여성보다 심장병 확률 약 2.44배
- ChestPainType : ASY(무증상)가 ATA보다 약 6배
- RestingECG : 차이 크지 않음 (ST 이상이 약간 높음)
- ExerciseAngina = Yes → 약 2.4배
- ST_Slope = Up → 심장병 확률 확연히 낮음

- 자주 쓴 코드 : `df.groupby('컬럼')['HeartDisease'].mean()` → 범주별 심장병 비율

4. 모델 비교 (전부 `train_test_split` 7:3, `random_state=42`)

- Pipeline 사용

- `make_column_transformer` + `OneHotEncoder` (범주형) → `make_pipeline(ct, model)`
- 목적 : 데이터 누수(data leakage) 방지

- Baseline : `DummyClassifier(strategy='constant', constant=1)` → **0.5942** (기준점)

- Scaler 효과 (`StandardScaler`)

모델	스케일 전	스케일 후
Logistic	0.8841	0.8804
LDA	0.8696	0.8696
SVM	0.7246	0.8841
KNN	0.7174	0.8841

→ 거리 기반 모델(SVM, KNN)은 스케일링 효과가 큼

- 앙상블 : AdaBoost 0.8659 / GradientBoosting 0.8768 / **RandomForest 0.8877** / ExtraTrees 0.8804
- Famous Trio : XGBoost 0.8297 / LightGBM 0.8732 / **CatBoost 0.8804** (기본 값 기준 CatBoost가 가장 우수)

5. CatBoost

- 특징
 - sklearn 호환 / 범주형 변수를 전처리 없이 직접 처리
 - 기본 목적함수 : Target 2개 값 → `Logloss` / 3개 이상 → `MultiClass`
- 핵심 코드
 - `np.where(X.dtypes != np.float)[0]` → 범주형 컬럼 인덱스 추출
 - `model.fit(X_train, y_train, cat_features=인덱스, eval_set=(X_test, y_test))`
 - `CatBoostClassifier(verbose=False, random_state=0)`
- 기본 CatBoost : **0.8804**

6. Optuna 튜닝

- 흐름 : `objective(trial)` 함수 정의 → 정확도 반환 → 최대화 탐색
- 핵심 메소드
 - `trial.suggest_categorical()` → 후보 중 선택
 - `trial.suggest_float()` / `suggest_int()` → 범위 내 탐색
 - `optuna.create_study(direction="maximize")` → 정확도 최대화
 - `study.optimize(objective, n_trials=50, timeout=600)` → 50회 시도 (최대 600초)
 - `study.best_trial` → `.value` / `.params` → 최적 결과 확인
 - `early_stopping_rounds=100` → 개선 없으면 조기 종료
- 튜닝 파라미터

파라미터	설명
objective	과적합 감지 / 최적 모델 선택 기준 (Logloss, CrossEntropy)
colsample_bylevel	학습 속도 향상, 품질엔 보통 영향 적음
depth	트리 깊이
boosting_type	Ordered / Plain (Ordered는 소규모 데이터에서 과적합 방지, 연산량 큼)

파라미터	설명
bootstrap_type	가중치 샘플링 방식 (Bayesian / Bernoulli / MVS)

- 조건부 파라미터 : Bayesian → `bagging_temperature` / Bernoulli → `subsample`
- 튜닝 후 : **0.8804** → **0.9094** (`classification_report` 로 precision / recall / f1 확인)
- 참고 : 노트북 내 Optuna 출력(best 0.9022, depth 12)과 최종 모델에 직접 넣은 값 (depth 10)은 서로 다른 실행 결과

7. Feature Importance

- `model.get_feature_importance()` → 변수별 중요도 → DataFrame → `px.bar` 로 시각화

8. 결론 (노트북 요약)

- EDA → 지표 선택 → 범주형 수치화 → **Pipeline으로 누수 방지** → 모델 비교 → CatBoost → **Optuna 튜닝으로 성능 개선** → Feature Importance 확인